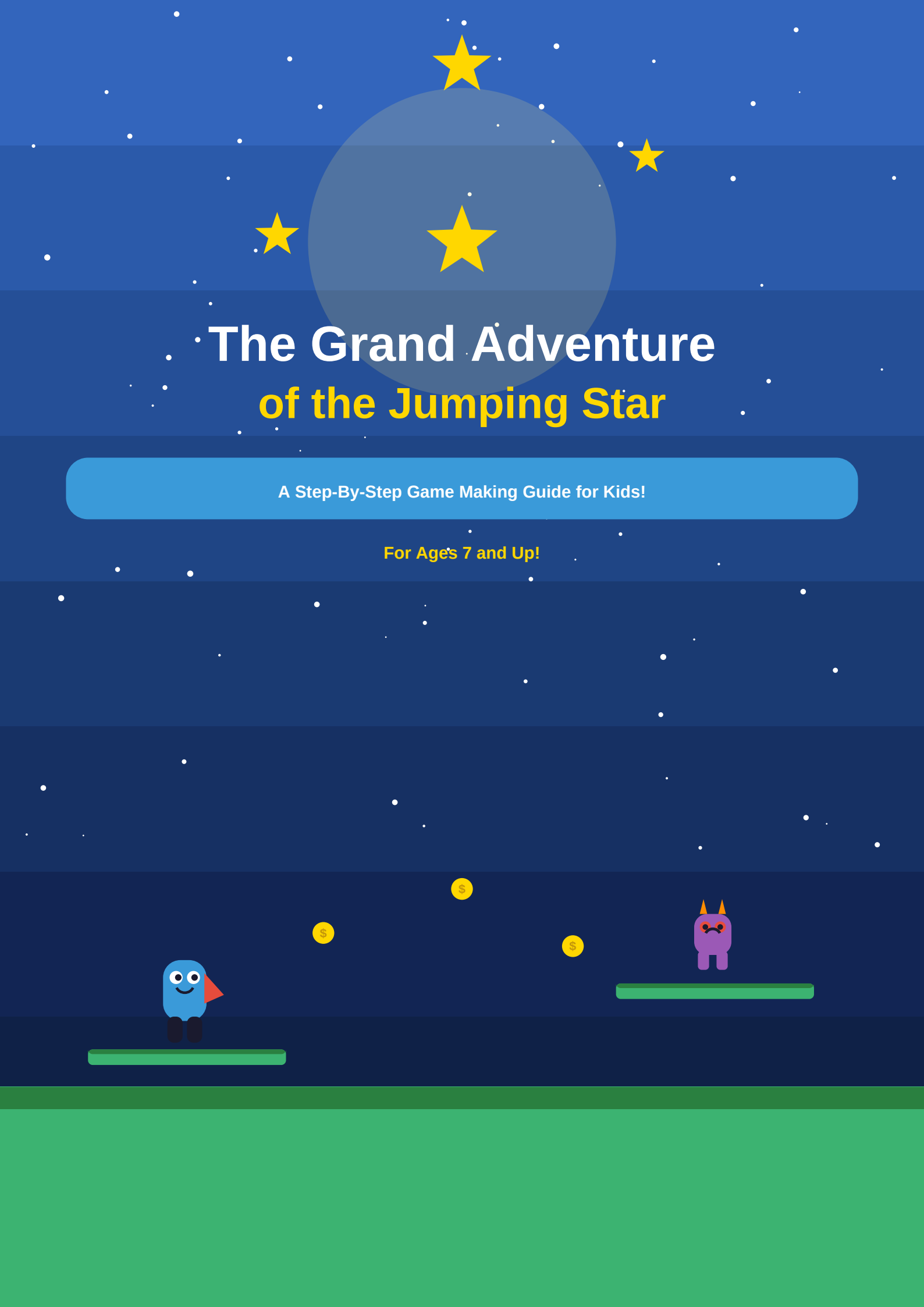
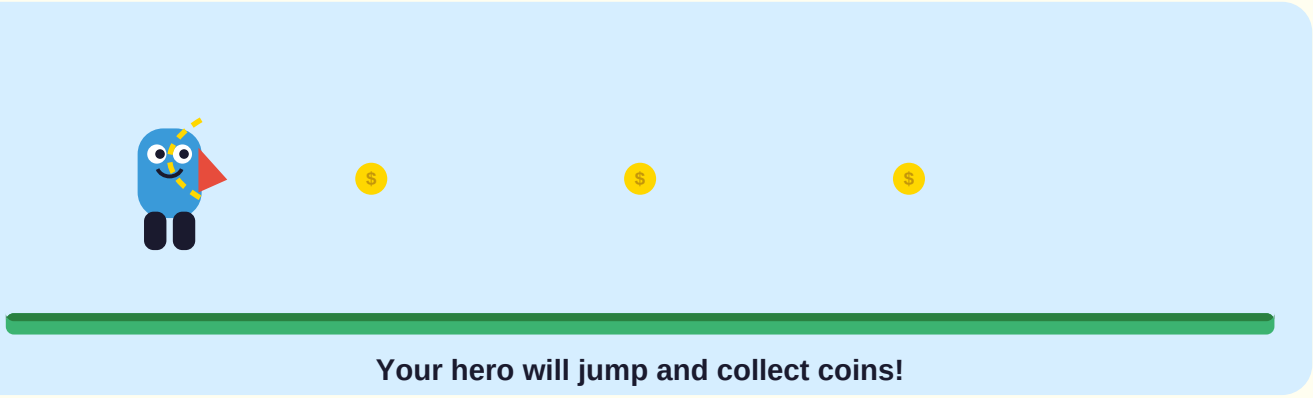

The Grand Adventure
of the Jumping Star
A Step-By-Step Game Making Guide for Kids!
For Ages 7 and Up!

Hi there, Game Wizard! :-)

Have you ever wanted to make your very own video game?

In this book, YOU are going to build a real game where a little hero jumps around, collects shiny coins, and reaches a secret door! It's called a **platformer** game.



Your hero will jump and collect coins!

We use a **FREE** program called **Godot** (say it like: *Goh-doh*). You will also write real computer code! Don't worry - the code in this book is short and easy. You can do this!

Ask a grown-up to help!

Some steps need a grown-up to download things or type on the keyboard. That is totally fine - great game developers always work in teams!

Platformer:

A type of game where you jump across platforms to reach the end. Think Mario or Sonic!

Godot:

A free program that helps you build games. It is your magic workshop!

GDScript:

The special language we use to tell Godot what to do. It is like giving instructions to a robot!

Ch.1 :-)

Open the Workshop

Let's get Godot on your computer!

Ready? Let's go!

This is the Godot workshop!

First, we need to put Godot on your computer. It is completely **free** and very quick!

1. Ask a grown-up to go to this website: godotengine.org
2. Click the big Download button.
3. Choose Godot Engine 4 - Standard for your computer type (Windows, Mac, or Linux).
4. The file is small - it downloads in about 1 minute!
5. Double-click the file you downloaded to open it. No installation needed!
6. A window called the Project Manager will pop up. That is the front door!

! **Use Godot 4!**

This book uses Godot 4. If you see a version number, make sure it starts with a 4. Godot 3 is different and the steps won't match.

Now let's create our game project!

1. Click the big New Project button.
2. Give your game a name - try JumpingStar (no spaces!).
3. Click Create and Edit.
4. The editor opens! This is where the magic happens.

The editor has lots of parts. You only need to know these four:

Panel Name	What it does
Scene Tree	Lists all your game objects
Inspector	Change the size, colour, and settings of things
Viewport	Your game stage - drag things here!
Script Editor	Where you type your magic code

***** **Save your work often!**

Press Ctrl + S (or Cmd + S on a Mac) to save after every step!

Ch.2 :-D

Meet Our Hero!

Build the jumping star!

Ready? Let's go!

Every game needs a hero! Our hero uses a special node called **CharacterBody2D**. It already knows how to walk, jump, and bump into things.

Node:

A building block in Godot. Everything in your game is made of Nodes stacked together - like LEGO!

CharacterBody2D:

A special Node that knows how to move around and bump into things. Perfect for a player!

1. In the Scene Tree (top left), click the + button or press Ctrl + A.
2. Type CharacterBody2D in the search box and press Enter.
3. Right-click the new node and choose Rename. Type: Player
4. Press Ctrl + A again. Add a Sprite2D - this is how the hero looks!
5. In the Inspector, find Texture and pick any colour you like.
6. Press Ctrl + A one more time. Add a CollisionShape2D - this is the hero's invisible armour!

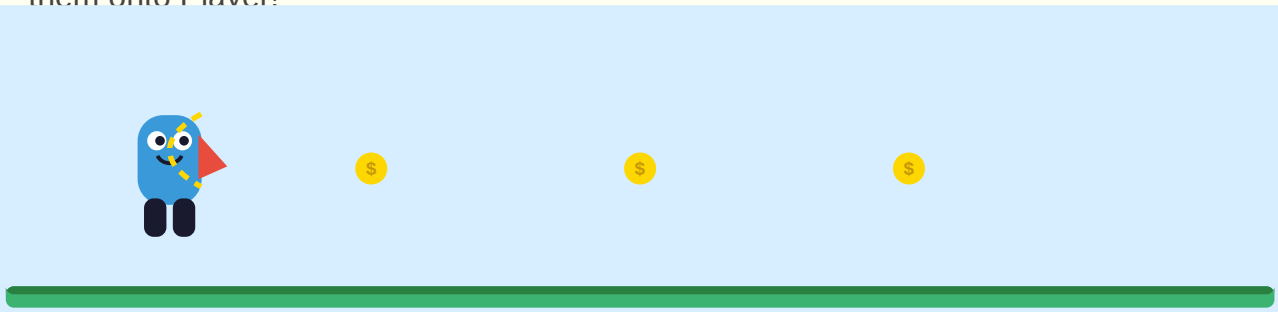
7 In the Inspector, set Shape to CapsuleShape2D.

8 Drag the orange handles to fit the capsule around your sprite.

9 Press Ctrl + S and save the scene as: Player.tscn

! **Very important!**

The Sprite2D and CollisionShape2D must go INSIDE the Player node. They should appear slightly indented underneath Player in the list. If they are at the same level, drag them onto Player!



Your hero will jump and collect coins!

Now we give the hero its moving spell - the code that makes it run and jump!

1 Click the Player node in the Scene Tree.

2 Click the small scroll icon at the top (it says Attach Script).

3 Click Create.

4 You will see a code editor. Delete everything that is already there.

5 Carefully type in the code below - or ask a grown-up to copy it in.

Player.gd - The Hero's Moving Spell

```
extends CharacterBody2D

# How fast the hero runs
const SPEED = 300.0
# How high the hero jumps (negative = upward in Godot)
const JUMP_VELOCITY = -450.0

# The hero's pockets
var coins_collected = 0
var health = 3

# This reads gravity from Godot automatically
var gravity = ProjectSettings.get_setting(
    "physics/2d/default_gravity")

func _physics_process(delta):
    # Pull the hero down when in the air
    if not is_on_floor():
        velocity.y += gravity * delta
    # Jump! Press Space or the A button
    if Input.is_action_just_pressed('ui_accept') and is_on_floor():
        velocity.y = JUMP_VELOCITY
    # Move left or right with the arrow keys
    var direction = Input.get_axis('ui_left', 'ui_right')
    if direction:
        velocity.x = direction * SPEED
    else:
        velocity.x = move_toward(velocity.x, 0, SPEED * delta * 10)
    move_and_slide()

func collect_coin():
    coins_collected += 1
    print("Coins: ", coins_collected)

func take_damage():
    health -= 1
    print("Ouch! Health left: ", health)
    if health <= 0:
        print("Game Over!")
        queue_free()
```

What are the # lines?

Lines starting with # are called comments. The computer ignores them! They are just notes to help you understand what the code is doing.

Ch.3 =]

Build the Ground!

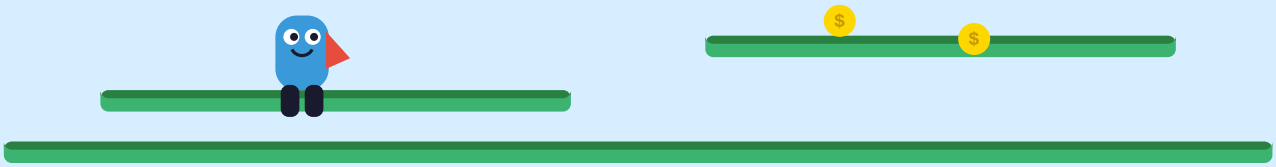
Make solid platforms to stand on

Ready? Let's go!

Our hero needs solid ground to land on! We use a **StaticBody2D** - a node that is solid and never moves. Perfect for floors!

StaticBody2D:

A solid object that never moves. Great for floors, walls, and platforms!



Solid platforms for your hero to land on!

1. Press Ctrl + N to start a brand new scene.
2. Click 2D Scene at the top. Rename the node to: Level
3. Press Ctrl + A and add a StaticBody2D. Rename it: Platform
4. Add a Sprite2D child to Platform. Make it green - that is your floor!
5. Add a CollisionShape2D child to Platform. Set Shape to RectangleShape2D.
6. Drag the handles to make it wide and flat.

7 Save this scene as: Level.tscn

Now let's put the hero into the level!

1 In Level.tscn, click the chain or link icon at the top of the Scene Tree.

2 Find Player.tscn and click Open. Your hero appears!

3 Drag the hero so they are floating just above the platform.

4 Press F5 to play the game!

What should happen?

The hero falls down and lands on the platform. Try pressing the arrow keys to run and SPACE to jump! If the hero falls through the floor, check that the Platform has its CollisionShape2D.

Ch.4 :-)

Shiny Coins!

Coins the hero can pick up

Ready? Let's go!

Time for coins! Coins are not solid - they just need to know when the hero touches them. We use an **Area2D** for this.

An Area2D is like an invisible bubble. When the hero walks inside it, something happens!

Area2D:

An invisible zone. When something walks in, it fires a Signal - like ringing a doorbell!

Signal:

A message Godot sends when something happens. Like the doorbell ringing when someone is at the door!

Touch a coin and it disappears - POOF!

1. Press Ctrl + N - new scene!
2. Add an Area2D node. Rename it: Coin
3. Add a Sprite2D child. Make it bright yellow and shiny!

4. Add a CollisionShape2D child. Set Shape to CircleShape2D.
5. Save as: Coin.tscn
6. Click the Coin node. Look for the Node tab next to the Inspector.
7. Double-click the signal called body_entered.
8. Click Connect. This wires up the doorbell!

Now attach a script to Coin and type this in:

Coin.gd - The Disappearing Spell

```
extends Area2D

# This runs when something walks into the coin
func _on_body_entered(body):
    # Check if it is the hero!
    if body.name == "Player":
        # Tell the hero they got a coin
        body.collect_coin()
        # Make the coin vanish - POOF!
        queue_free()
```

1. Open Level.tscn.
2. Use the chain icon to add Coin.tscn to the level.
3. Press Ctrl + D to copy and paste it - make lots of coins!

4. Place them floating above platforms so the hero can reach them.

5. Press F5 to test. Run through the coins and watch them disappear!

* **Check the Output panel!**

When you collect a coin, look at the bottom of the screen. You should see 'Coins: 1' appear! That is your code working!

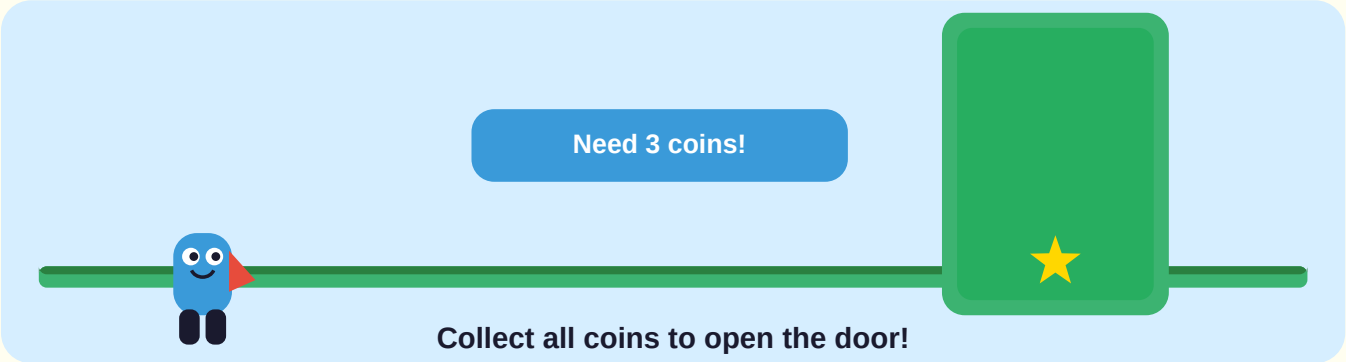
Ch.5 :-D

The Magic Door!

A goal that checks your coins

Ready? Let's go!

Our adventure needs an ending! Let's make a big green door that only opens when the hero has collected ALL the coins.



1. Press Ctrl + N - new scene!
2. Add an Area2D node. Rename it: Goal
3. Add a Sprite2D child. Make it big and bright green - like a door!
4. Add a CollisionShape2D child. Use RectangleShape2D and make it tall like a door.
5. Save as: Goal.tscn
6. Click Goal, go to the Node tab, and connect the body_entered signal.

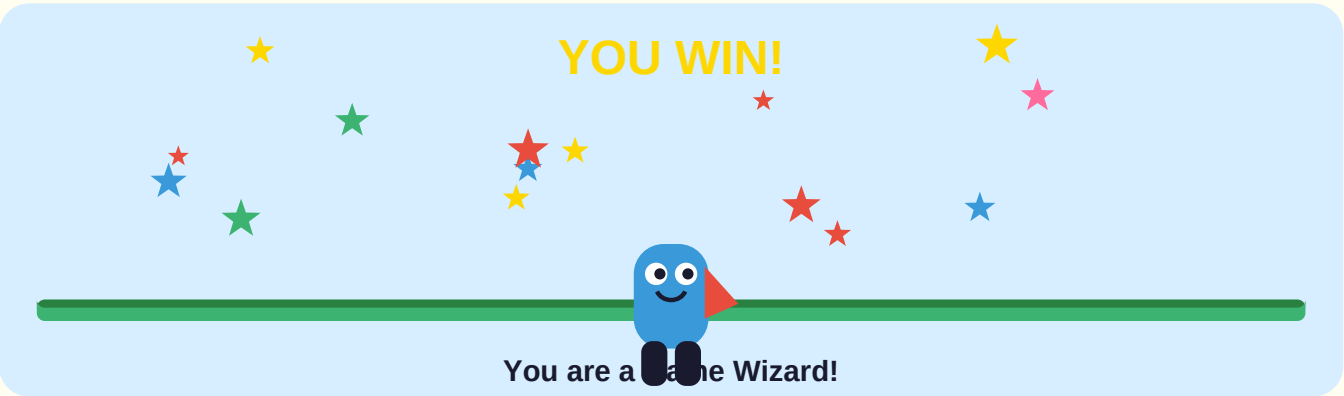
Goal.gd - The Door Spell

```
extends Area2D

# Change this number to match how many coins you placed!
@export var required_coins = 3

func _on_body_entered(body):
    if body.name == "Player":
        if body.coins_collected >= required_coins:
            print("YOU WIN! Great job!")
            get_tree().quit()
        else:
            var need = required_coins - body.coins_collected
            print("You need ", need, " more coins!")
```

! **Do not forget this step!**

After placing Goal in Level.tscn, click on the Goal node. In the Inspector, find Required Coins and set it to the EXACT number of coins you placed. If the number is wrong, the door will never open!

1. Open Level.tscn and add Goal.tscn with the chain icon.
2. Place the door at the far end of the level.
3. Click the Goal node. In the Inspector, set Required Coins to match your coins.
4. Press F5 and test! Go to the door WITHOUT all the coins - it should stay locked.
5. Now collect every coin and run to the door - YOU WIN!


YOU WIN!
You are a [illegible]e Wizard!

Ch.6

:-O A Beautiful Sky!

Make the background look amazing

Ready? Let's go!

Right now everything looks flat. Let's add a sky that moves!

This trick is called **Parallax Scrolling**. Far-away things move slowly and close things move fast. This makes the world look really deep and huge!

Parallax Scrolling:

A trick where far things move slowly and close things move fast. It makes flat games look like they have real depth!

1. Open Level.tscn.
2. Press Ctrl + A and add a ParallaxBackground. Rename it: Skybox
3. IMPORTANT: Drag Skybox to be the FIRST child in the list - above everything else!
4. Add a ParallaxLayer child to Skybox. Rename it: FarClouds
5. Add a Sprite2D to FarClouds. Give it a light blue colour and stretch it to fill the screen.
6. Click FarClouds. In the Inspector find Motion and set Scale X to 0.1
7. Add another ParallaxLayer to Skybox. Rename it: NearHills

8 Add a Sprite2D to NearHills. Make it dark green, placed near the ground.

9 Click NearHills. Set Scale X to 0.5

10 For BOTH layers, set Motion > Mirroring > X to 1024 so they loop forever.

11 Press F5 and run! The clouds move slowly and the hills move faster!

*** What do the numbers mean?**

Scale X = 0.1 means the clouds move only 10% as fast as the camera. Scale X = 0.5 means the hills move 50% as fast. Try different numbers!

Ch.7 :-D

Paint Giant Levels!

Use TileMap to draw levels super fast

Ready? Let's go!

Placing platforms one at a time is slow. What if you want a really big level?

We use a **TileMap**! It works like a stamp kit - you pick a block and paint it onto the level super fast by clicking and dragging!

TileMap:

A special node that lets you paint your level by clicking and dragging. Super fast!

TileSet:

Your collection of stamps. Each stamp is one block you can paint with.

1. Open Level.tscn. Delete the old Platform if you still have it.
2. Press Ctrl + A and add a TileMap. Rename it: GroundPainter
3. Click GroundPainter. In the Inspector, click Tile Set then [empty] > New TileSet.
4. Click the TileSet resource. A TileSet Editor panel opens at the bottom.
5. In the TileSet Editor, click + and choose New Atlas.
6. Pick an image or colour for your block, then click Add Single Tile.

7 You now have a stamp!

! **Add armour to your tile or the hero will fall through!**

Click your tile in the TileSet Editor. Click the Physics tab. Click + to add a Physics Layer. Draw a blue box over the whole tile. This invisible armour makes the tile solid!

1 Go back to the 2D view by clicking the 2D tab at the top.

2 Select GroundPainter in the Scene Tree.

3 Click your stamp in the TileMap panel at the bottom.

4 Click and drag in the middle of the screen to PAINT your level!

5 Make platforms, walls, and floating islands.

6 Press F5 and run across your painted world!

* **Make it really big!**

You can now make a level as big as you want! Try painting long paths with gaps to jump over and tall walls to climb.

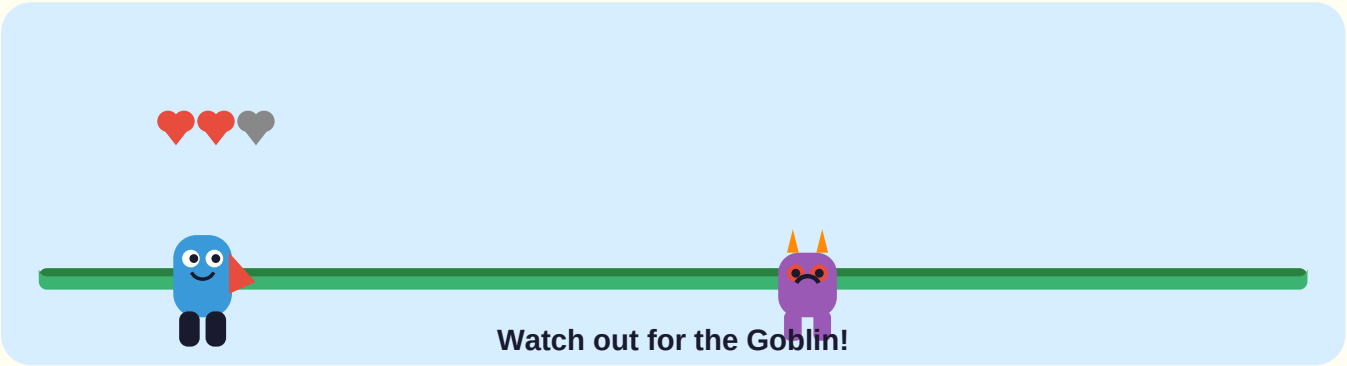
Ch.8 >:-)

Enemy Goblins!

Add a marching enemy and health

Ready? Let's go!

Our hero is too safe! Let's add an **Enemy Goblin** that marches back and forth. If the hero touches it - ouch!



Watch out for the Goblin!

1. Press Ctrl + N - new scene!
2. Add a CharacterBody2D. Rename it: Goblin
3. Add a Sprite2D child. Make it purple or red - scary colours!
4. Add a CollisionShape2D child. Use CapsuleShape2D.
5. Save as: Goblin.tscn

Goblin.gd - The Marching Spell

```
extends CharacterBody2D

const SPEED = 100.0
# 1.0 = march right, -1.0 = march left
var direction = 1.0

# Goblin needs gravity too or it will float!
var gravity = ProjectSettings.get_setting(
    "physics/2d/default_gravity")

func _physics_process(delta):
    if not is_on_floor():
        velocity.y += gravity * delta
    velocity.x = direction * SPEED
    # Turn around when the goblin hits a wall
    if is_on_wall():
        direction *= -1.0
    # Flip the sprite to face the right way
    $Sprite2D.flip_h = direction < 0.0
    move_and_slide()

# Called when the hero bumps the HurtBox
func _on_hurt_box_body_entered(body):
    if body.name == "Player":
        body.take_damage()
```

1. In Goblin.tscn, press Ctrl + A and add an Area2D child to the Goblin. Rename it: HurtBox
2. Add a CollisionShape2D to HurtBox. Make it a little BIGGER than the goblin's shape.
3. Click HurtBox. Go to the Node tab and double-click body_entered.
4. Connect it to Goblin.gd to link the attack spell.
5. Open Level.tscn and add Goblin.tscn with the chain icon.

6 Place the Goblin on a long platform.

7 Press F5 and test! The goblin should march. Walk into it and watch the health count drop!

* **What happens when health reaches 0?**

When health reaches 0, the take_damage() spell calls queue_free(). This removes the hero from the game and you see 'Game Over!' in the Output panel.

Ch.9 :-)

Sounds and Music!

Make your game come alive with audio

Ready? Let's go!

A game without sounds is like a birthday cake without candles! Let's add a coin sound, a hurt sound, and background music!

AudioStreamPlayer:

A Godot node that plays a sound file. Add it anywhere and call .play() to make it go!

1. Ask a grown-up to help you find a free coin sound (.wav or .ogg file) at freesound.org
2. Also find a short hurt or ouch sound.
3. And find a looping music track at opengameart.org - both sites are free!
4. Drag all three audio files into the FileSystem panel in Godot (bottom left). Godot imports them automatically!

Add background music to the level:

1. Open Level.tscn.
2. Press Ctrl + A and add an AudioStreamPlayer. Rename it: Music
3. In the Inspector, drag your music file onto the Stream property.

4 Tick the Autoplay box so music starts as soon as the level loads.

Add coin and hurt sounds to the hero:

1 Open Player.tscn.

2 Add an AudioStreamPlayer child. Rename it: CoinSound. Assign your coin audio to its Stream.

3 Add another AudioStreamPlayer. Rename it: HurtSound. Assign your hurt audio to its Stream.

Player.gd - Add these lines to make sounds play

```gdscript
# Find the collect_coin function and add the .play() line:
func collect_coin():
    coins_collected += 1
    print("Coins: ", coins_collected)
    $CoinSound.play()      # ADD THIS LINE

# Find the take_damage function and add the .play() line:
func take_damage():
    health -= 1
    print("Ouch! Health: ", health)
    $HurtSound.play()      # ADD THIS LINE
    if health <= 0:
        print("Game Over!")
        queue_free()
```

* **Test your sounds!**

Press F5. Grab a coin - you should hear a ping! Walk into a goblin - you should hear an ouch! Music should play the whole time.

YOU DID IT! :-D

You have built a **real game** with all of this!

- **A hero that runs and jumps**
- **Coins that disappear when collected**
- **A door that checks your coins**
- **A beautiful scrolling sky**
- **A level you painted yourself**
- **A marching enemy goblin**
- **A health system**
- **Music and sound effects**

What can you try next?

Add more levels: Change get_tree().quit() to get_tree().change_scene_to_file("res://Level2.tscn")

Make it look amazing: Replace colour rectangles with real pixel art using AnimatedSprite2D

More enemies: Copy your Goblin and make some that jump or move faster!

Share with friends: Ask a grown-up to help with Project > Export to share your game

Keep making games. You are a Game Wizard!